

The Automaton Underneath: The Additive Input Pathway Is a Parasitic Attractor for State Tracking in Householder Linear RNNs

Gunner Levi Howe
gunnerlevihowe@gmail.com

July 2026

Abstract

Linear RNNs with input-dependent Householder-product transitions (DeltaNet/DeltaProduct-class) can provably represent hard state-tracking automata, yet trained models notoriously fail to length-generalize — a gap recent work names “expressivity vs. learnability” and explicitly attributes to optimization, without a causal account. We give one, in a pre-registered, within-architecture causal ablation: the same model, with one term deleted — the additive input injection $b_t = W_b e_t$. With b_t , models fit length 32 perfectly and collapse out-of-distribution on parity, S_4 , A_5 , and the non-solvable S_5 word problems (median position-512 accuracy 0.20 on S_5). Without b_t — input acting *only* through the orthogonal transitions — the same architecture at the same width learns the *exact* automaton: median accuracy 1.00 at $16\times$ the training length, at every width admitted by a representation law we state and test: the minimal number of Householder factors per token equals the maximal reflection length of the task’s generators in the representation the task format pins (parity 1, S_4 3, A_5 and S_5 4); below it, $-b$ models cannot fit at all, and the contrast with DeltaProduct’s S_4/A_5 -at- $n_h=2$ (group-element classification, $SO(3)$ realization) shows the law is representation-relative rather than absolute. Two pre-registered discriminators locate the mechanism. (i) Initialized *at* a verified-exact solution with $W_b=0$, Adam grows the additive path (probe $\|W_b e\|$ $0 \rightarrow 0.77$ on S_5) and pulls the model off the exact solution (0.23 at position 512) while $-b$ controls stay at 1.00 — the additive route is an attractor, not merely a basin around random inits. (ii) Our pre-registered prediction that the shortcut’s in-domain fit *lives on* b_t was wrong, and the registered kill criterion fired: zeroing W_b at inference leaves in-domain fit intact in all 49 fitting seeds — and at the law-minimal width it *restores exact length generalization* (parity 5/5, S_5 5/5, S_4 4/5, A_5 4/5 seeds). The additive pathway is parasitic: a norm-inflation channel that cross-entropy feeds, which first destabilizes and then conceals a correctly learned automaton. Boundaries are reported with the same discipline: above-law widths restore in only 1–2/5 seeds, and the cyclic mod-3 task drifts rather than locking even without b_t — killing our own earlier “transposition anchor” hypothesis, since all-even-generator A_5 locks exactly. All 202 runs, predictions, and kill criteria were pre-registered in-repository before any grid run; every number regenerates from run artifacts.

1 Introduction

State tracking — maintaining the running state of an automaton while reading a sequence — is the sharpest known separation between attention/parallel architectures and true recurrence [6, 2], and the leading sub-quadratic architectures (DeltaNet-class linear RNNs with input-dependent orthogonal or near-orthogonal transitions) were designed in part to close it [4, 10]. A striking 2025–2026 literature documents that they close it *in principle but not in training*: models that provably contain exact solutions fit the training length and collapse beyond it [9, 7, 1, 2]. Shakerinava et al. [9] name the gap — “the main bottleneck . . . is not expressivity but optimization” — demonstrate it survives near-solution initialization in diagonal SSMs, and call for a loss-landscape account. Cross-architecture evidence points at the input pathway: Terzić et al. [13] report that diagonal selective SSMs *without* the additive B matrix length-generalize better on commutative automata; Ebrahimi and Memisevic [3] show pure bilinear (non-affine) RNNs length-generalize

on parity and random permutation FSMs where affine variants fail; and injection-free orthogonal-transition architectures extrapolate spectacularly [12, 5]. But none of these is a *causal* test: each compares different architectures (or removes capacity along with the pathway, as in diagonal SSMs, which lose non-commutative expressivity with B), none explains *why* gradient descent prefers the failing solution, and PD-SSM’s exact S_5 extrapolation *with* an additive term [14] falsifies any architecture-universal reading.

This note supplies the missing experiment, pre-registered end-to-end. In a fixed DeltaProduct-style Householder linear RNN — where the orthogonal transitions alone are expressive enough for every task studied, so deleting the additive injection b_t changes *what SGD finds* but not *what the model can represent* — we delete exactly one term and observe, across five seeds per cell:

1. **The flip** (§3): $+b$ models fit length 32 and collapse (shortcut); $-b$ models learn the exact automaton to length 512 ($16\times$), including the non-solvable S_5 , at matched width, budget, and data.
2. **A representation law with a necessity side** (§4): the minimal n_h that length-generalizes equals the maximal generator reflection length $\text{rank}(I - P)$ in the representation pinned by the task format (parity 1, S_4 3, A_5/S_5 4); every below-law $-b$ cell is no-fit. DeltaProduct’s S_4/A_5 at $n_h=2$ under group-element classification [10] is thereby reconciled, not contradicted: the law is representation-relative, and the task format selects the representation.
3. **The mechanism** (§5): from a verified-exact initialization with $W_b=0$, training *grows* the additive path and destroys length generalization while $-b$ controls are stable — and our registered prediction that the fit routes through b_t was falsified by its own kill criterion: the additive path is *parasitic*. At law-minimal width the exact automaton is learned *underneath* it, and zeroing W_b at inference — no retraining — restores exact generalization in 18/20 seeds across parity/ S_4 / S_5 / A_5 .

2 Setup

Model. A single recurrent layer with input-dependent Householder-product transitions [10]: $h_t = R_{n_h}(x_t) \cdots R_1(x_t) h_{t-1} + b_t$, where each $R_i(x_t) = I - 2u_i u_i^\top$ (u_i a learned, normalized, input-dependent vector), $d=112$, $b_t = W_b e_t$ is the standard additive injection, and a per-step *linear* readout predicts the automaton state. The $-b$ ablation deletes b_t only: input then acts exclusively by selecting orthogonal transitions — the literal automaton form. Training: full 32-length sequences, 40 epochs, Adam, identical budget in every cell (Appendix A); no cell-specific tuning anywhere.

Tasks. Per-step tracking of a designated element’s position under generator sequences (vocabulary = 2 generators): parity ($\mathbb{Z}_2$), mod-3 ($\mathbb{Z}_3$), S_4 (transposition + 4-cycle), A_5 (3-cycle + 5-cycle; all-even), S_5 (transposition + 5-cycle; non-solvable). Group closures and generator reflection lengths are asserted programmatically at runner start. Train $L=32$, evaluate per-position to $L=512$; **GEN** = median position-512 accuracy >0.9 ; **shortcut** = fits position 32 (>0.9) but <0.6 at 512; **no-fit** otherwise. Five seeds per primary cell; medians with [min, max].

Pre-registration. Grid, predictions, kill criteria, and decision rules were committed to the repository *before any grid run existed* (commit 2cf13e5), with verdicts recorded against them unchanged (84e21c8); the one exploratory antecedent (a 3-seed June pilot of the parity/ S_5 flip) is disclosed as such, and all its cells were re-run from scratch under this harness. Every number in this paper is a macro regenerated from the 202 run artifacts and byte-verified (Appendix A).

3 One deleted term flips shortcut to exact automaton

Table 1 is the paper’s central result; Figure 1 shows the per-position curves. With the additive path, every task that fits at all follows the shortcut profile: perfect at the training length, collapsed by $2\text{--}4\times$ beyond it (S_5 : 1.00 at position 32 vs. 0.20 at 512). Deleting b_t flips every such cell at or above the law width to exact generalization: parity 1.00, S_4 1.00, A_5 1.00, S_5 1.00 at position 512 (medians of 5; S_5 and parity [1.00, 1.00]). The GRU reference generalizes everywhere (as known); the negative-eigenvalue diagonal model cannot fit S_5 (no order-5 real eigenvalue), reproducing the expressivity wall the Householder parameterization exists to cross.

task	n_h	+b (median pos-32 / pos-512)		-b (pos-32 / pos-512)		+b, $W_b:=0$ at inference	
		fit	OOD	fit	OOD	pos-512	seeds GEN
parity	1	1.00	0.50	1.00	1.00	1.00	5/5
mod-3	2	1.00	0.33	1.00	0.34	0.34	2/5
S_4	2	0.26	0.24	0.27	0.24	—	—
S_4	3	1.00	0.26	1.00	1.00	0.99	4/5
A_5	3	0.21	0.19	0.21	0.20	—	—
A_5	4	1.00	0.22	1.00	1.00	1.00	4/5
S_5	3	0.28	0.19	0.27	0.20	—	—
S_5	4	1.00	0.20	1.00	1.00	0.99	5/5

Table 1: Core grid (medians over 5 seeds; train $L=32$, test $L=512$; full per-cell ranges in the artifacts). At and above the law width, $+b$ cells shortcut, $-b$ cells are exact; below it (rows $S_4@2$, $A_5@3$, $S_5@3$) both fail to fit — the necessity side of the law. Right columns: zeroing W_b at inference on the trained $+b$ model restores generalization at law-minimal widths.

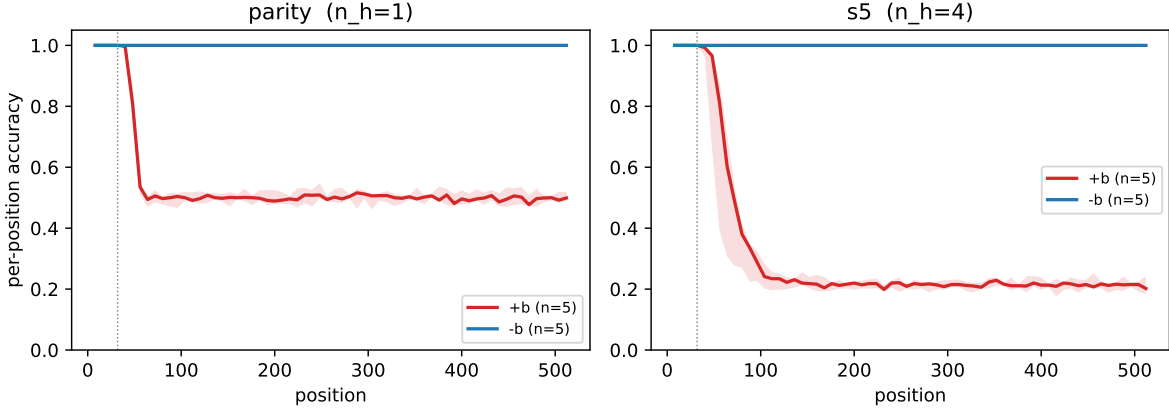


Figure 1: Per-position accuracy, median (band = seed min-max), train $L=32$ (dotted). The same architecture at the same width: $+b$ (red) fits and collapses; $-b$ (blue) is exact to $16\times$ the training length, including non-solvable S_5 .

4 The law: minimal n_h = generator reflection length, representation-relative

For a permutation generator P , its reflection length in the defining representation is $\text{rank}(I - P)$: a transposition is 1 reflection, a 4-cycle 3, a 5-cycle 4. Because each step must realize a generator as a product of n_h Householder factors, representability requires $n_h \geq \max_g \text{rank}(I - P_g)$ in whatever faithful representation the model realizes. Empirically (Fig. 2), the $-b$ models obey this law exactly in its defining-representation form: minimal generalizing $n_h = 1$ (parity), 3 (S_4), 4 (A_5), 4 (S_5), and every below-law cell is no-fit — e.g. S_5 at $n_h=3$: 0.27 at position 32. For S_5 the necessity is representation-independent: in every faithful representation of S_5 the 5-cycle’s reflection length is 4 (checked across the irreducible representations), so $n_h=4$ is a true lower bound, which the $n_h=3$ cell confirms behaviorally.

For S_4 and A_5 , however, Siems et al. [10] report robust extrapolation at $n_h=2$ via their isomorphisms to rotation groups in $SO(3)$, where every element is a product of ≤ 2 reflections. Our S_4/A_5 cells at $n_h=2$ (and A_5 at 3) are no-fit — both results are correct, and the reconciliation is the point: their format (group-element classification from a matrix state) admits the compact $SO(3)$ realization; our format (point tracking through a vector state with a per-step linear readout) pins the defining permutation representation, e.g. because the $SO(3)$ realization of S_4 carries the four objects as unsigned diagonal axes whose sign ambiguity a linear readout cannot resolve. The law that survives both datasets is representation-relative:

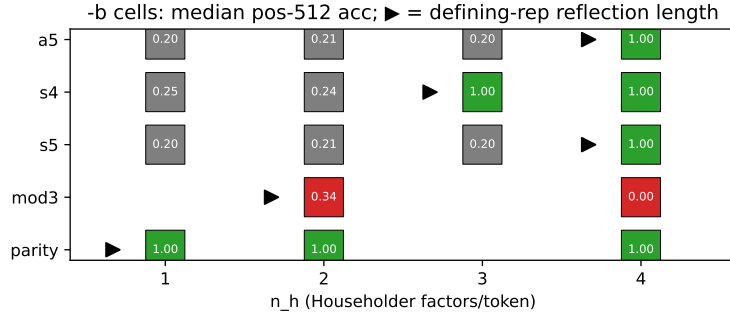


Figure 2: $-b$ cells: median position-512 accuracy per (task, n_h); $\blacktriangleright$ marks the defining-representation generator reflection length. Generalization begins exactly at the law width (green), below it nothing fits (gray); mod-3 (drift, §5) is the labeled exception.

minimal per-step compute equals the generators’ reflection length in the cheapest faithful representation the task format admits. Task format is thus a hidden variable in state-tracking benchmarks — two papers can disagree about “the n_h needed for S_4 ” while both being right.

5 Mechanism: a parasitic attractor, discriminated by pre-registered arms

The additive route attracts even from the exact solution (E3). We construct exact solutions by hand (reflection programs whose composed factors are asserted equal to the generator permutation matrices; dead-axis factors implement identity), verify them (pre-training accuracy 1.00 at every probe position — a registered instrument gate), set $W_b=0$, and train with the standard budget. Across 5 seeds on both parity and S_5 : the probe additive-path magnitude $\|W_b e\|$ grows steadily from 0 to 0.41 (parity) and 0.77 (S_5), the state norm inflates (impossible through the norm-preserving reflections), and position-512 accuracy falls to 0.50/0.23 — while the $-b$ exact-init controls remain at 1.00/1.00 (Fig. 3). The additive pathway is not merely a basin that captures random initializations; under Adam + cross-entropy it destabilizes the exact global solution itself. A natural reading, stated as interpretation: orthogonal transitions cannot grow $\|h\|$, so the additive path is the only channel through which cross-entropy’s appetite for larger logits can express itself, and what it adds is position-dependent drift that is harmless in-domain and fatal out-of-distribution.

Our registered mechanism prediction was wrong; the kill criterion fired; the truth is better (P3/K2). We pre-registered that the $+b$ shortcut’s in-domain fit *routes through* the additive path (prediction: zeroing W_b at inference collapses even position-32 accuracy). It does not — of the 49 seeds across all $+b$ cells that fit the training length, 49 retain in-domain accuracy under $W_b:=0$ (e.g. S_5 : 1.00 at position 32), so K2 fired as registered and the “leaky-counter route” story is withdrawn. What replaces it is stronger: at the law-minimal width, zeroing W_b at inference *restores exact length generalization* — parity 1.00 (5/5 seeds), S_5 0.99 (5/5), S_4 4/5, A_5 4/5. The trained “shortcut” model at minimal width *contains the exact automaton*: the reflections converge to it anyway, and the additive path merely conceals it. The same holds for the exact-init arms after being pulled off ($W_b:=0$ restores 1.00/1.00). The additive pathway is therefore *parasitic* in the precise sense that it neither performs the fit nor helps it; it grows because it is the cheapest confidence channel, and its growth both destabilizes training away from exact solutions and masks exact solutions once learned.

Boundaries (reported, not smoothed). The concealment result is width- and task-scoped: above the law width the freed reflection capacity co-adapts with the parasite and $W_b:=0$ restores only 1–2/5 seeds on parity ($n_h=2, 4$); per-seed exceptions exist at law width (S_4 4/5, A_5 4/5). And mod-3 never locks: even $-b$, its median position-512 accuracy craters (0.00 at $n_h=4$ — systematic rotational drift, worse than chance, with individual seeds at 1.00). Notably this *kills our own prior hypothesis* (from the June pilot) that a transposition generator is what anchors exact solutions: A_5 ’s generators are all even — no transposition,

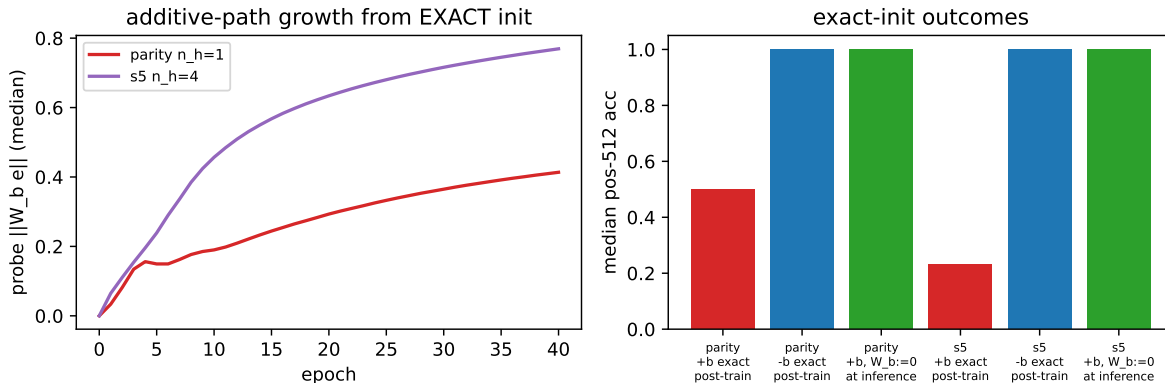


Figure 3: E3, medians over 5 seeds. Left: from a verified-exact initialization with $W_b=0$, training grows the additive path. Right: outcomes — the $+b$ exact-init model is pulled off the exact solution; the $-b$ control is stable; zeroing W_b at inference restores it.

no reflection among them — yet A_5 locks exactly (1.00). What separates the locking tasks from mod-3 is open; we note only that mod-3 is abelian with a single continuously embeddable rotation generator, while the locking tasks force non-commuting discrete constraints.

Relation to rival mechanism accounts. The unexplored-states hypothesis [8] attributes length-generalization failure to training-time state coverage; our exact-init arms hold the initial solution (and hence its reachable state distribution) fixed and still observe the pull-off, so state coverage is not the operative variable here. Error-control theory [1] proves affine trackers accumulate uncorrectable error at inference; consistently, our $+b$ models fail OOD — but the same weights with $W_b:=0$ are exact, and training *grows* b_t from zero at an exact solution, so in this family the affine term is not a fixed design constraint that error dynamics must cope with but a *learned* liability that optimization actively acquires. The two accounts are complementary to ours (inference-time and data-distribution, respectively); the within-architecture ablation locates the training-time cause.

6 Discussion

What this changes. The expressivity-learnability gap in Householder linear RNNs has a locatable, causal, single-term source in this setting — and the failure it produces is not absence of the solution but *occlusion* of it. That reframes the practical question from “how do we make these models learn state tracking” to “how do we stop one pathway from first hijacking and then hiding what they already learn.” The minimal demonstration also yields an unreasonably cheap diagnostic: evaluating a trained model with its additive input path zeroed is a one-line probe for a concealed automaton, and at minimal width it doubles as a repair (18/20 seeds across the four group tasks) — with the honest caveat that per-seed exceptions exist and the trick degrades off the minimal width.

What this does not claim. No universality: PD-SSM achieves exact S_5 extrapolation with an additive term in a different architecture [14], and reveal-style supervision makes the additive path load-bearing and helpful [11]; our claims are scoped to Householder-product linear RNNs under standard supervision, where the transitions alone are expressive enough. No language-modeling prescription: in an LM the additive pathway carries content, and these tasks use input only to select transitions; “delete W_b ” is a probe and a mechanism demonstration, not a training recipe. The law’s necessity side is representation-independent only where representation theory makes it so (S_5); elsewhere it is conditional on the format-pinned representation, which is exactly the point.

7 Limitations

Single architecture family, optimizer, and budget; $d=112$; five tasks with two-generator vocabularies; 5 seeds per cell (counts over p -values throughout); point-tracking format with per-step linear readout. Pre-registration is repository-native (commit-stamped in our own history, hashes in §2 and Appendix A), not third-party. The June pilot preceded registration and is disclosed; its mod-3 “flip” did not survive 5 seeds (median craters; reported in §5). The parasite account’s scope beyond this family — and what separates locking tasks from drifting ones — are open.

Acknowledgments

This note answers, in a minimal system, the loss-landscape question posed by Shakerinava et al. [9], and builds directly on the DeltaProduct parameterization of Siems et al. [10].

References

- [1] Jiwan Chung, Heechan Choi, and Seon Joo Kim. Rethinking state tracking in recurrent models through error control dynamics. *arXiv preprint arXiv:2605.07755*, 2026.
- [2] Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the Chomsky hierarchy. In *International Conference on Learning Representations*, 2023. arXiv:2207.02098.
- [3] M. Reza Ebrahimi and Roland Memisevic. Revisiting bi-linear state transitions in recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2505.21749.
- [4] Riccardo Grazi, Julien Siems, Arber Zela, Jörg K.H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations*, 2025. arXiv:2411.12537.
- [5] Jeonghoon Lee. A held-out transition-pair falsifier for long-horizon non-abelian state tracking. *arXiv preprint arXiv:2606.07254*, 2026.
- [6] William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models. *International Conference on Machine Learning*, 2024. arXiv:2404.08819.
- [7] Mayank Mishra, Shawn Tan, Ion Stoica, Joseph Gonzalez, and Tri Dao. M²RNN: Non-linear RNNs with matrix-valued states for scalable language modeling. *arXiv preprint arXiv:2603.14360*, 2026.
- [8] Ricardo Buitrago Ruiz and Albert Gu. Understanding and improving length generalization in recurrent models. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2507.02782.
- [9] Mehran Shakerinava, Behnoush Khavari, Siamak Ravanbakhsh, and Sarath Chandar. The expressive limits of diagonal SSMs for state-tracking. *arXiv preprint arXiv:2603.01959*, 2026.
- [10] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. DeltaProduct: Improving state-tracking in linear RNNs via Householder products. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2502.10297.
- [11] Julien Siems, Riccardo Grazi, Korbinian Pöppel, Kirill Kalinin, Hitesh Ballani, and Babak Rahmani. Learning state-tracking from code using linear RNNs. *arXiv preprint arXiv:2602.14814*, 2026.
- [12] Ilmo Sung. Robust reasoning as a symmetry-protected topological phase. *arXiv preprint arXiv:2601.05240*, 2026.
- [13] Aleksandar Terzić, Michael Hersche, Giacomo Camposampiero, Thomas Hofmann, Abu Sebastian, and Abbas Rahimi. On the expressiveness and length generalization of selective state-space models on regular languages. In *AAAI Conference on Artificial Intelligence*, 2025. arXiv:2412.19350.

- [14] Aleksandar Terzić, Nicolas Menet, Michael Hersche, Thomas Hofmann, and Abbas Rahimi. Structured sparse transition matrices to enable state tracking in state-space models. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2509.22284.

A Reproducibility

Pre-registration (grid, predictions P-R0/E1/E2/E3/P3, kill criteria K0–K2, decision rules) is commit `2cf13e5`, made before any grid artifact existed; verdicts against it, unchanged, are commit `84e21c8`. The grid is 202 runs (`statetrack_note/runs/`, one JSON artifact each, git-hash-stamped): 5 seeds per primary cell, 3 for references. Runner (`statetrack_note/runner.py`) asserts group closures, generator reflection lengths, and exact-construction correctness programmatically before running; it is idempotent. Every number above is a macro in `numbers.tex` generated by `analyze.py` from the artifacts and byte-verified by `verify_regen.py`. Model/training configuration: $d=112$, embeddings and readout linear, 40 epochs, batch 512, $n=4000$ sequences of length 32, Adam (3×10^{-3}), per-position cross-entropy; evaluation $n=1000$ at $L=512$. Hardware: one RTX 3080; full grid ≈ 53 minutes.